

Laporan Praktikum Minggu Ke-3 Kontrol Cerdas

Minggu ke-3

Nama : Ika Putri Ayuningtia

NIM : 224308057

Kelas : TKA 7C

Akun Github (Tautan) : <https://github.com/ikaputriayuningtia-sketch>

1. Judul Percobaan

Image Classification with CNN

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami konsep dasar Deep Learning dalam sistem kendali.
2. Mengimplementasikan Convolutional Neural Network (CNN) untuk klasifikasi objek.
3. Menggunakan TensorFlow dan Keras untuk membangun model Deep Learning.
4. Mengintegrasikan model CNN dengan Computer Vision untuk deteksi objek secara real-time.
5. Menggunakan dataset dari Kaggle untuk pelatihan model.
6. Mengembangkan mode Night Vision untuk deteksi objek dalam kondisi pencahayaan rendah.

3. Landasan Teori

A. Machine Learning dalam Sistem Kendali

Deep Learning dalam Sistem Kendali

Deep Learning merupakan bagian dari *machine learning* yang menggunakan jaringan saraf tiruan dengan banyak lapisan untuk mempelajari pola dari data. Teknologi ini memiliki kemampuan untuk mengekstraksi ciri (feature) secara otomatis sehingga sangat bermanfaat pada sistem kendali cerdas, seperti kendaraan

otonom, robotika, sistem pengawasan, hingga *quality control* industri (Sutrisno, 2020). Dalam sistem kendali, penerapan Deep Learning memungkinkan:

- Kendaraan otonom untuk mengenali rambu lalu lintas dan objek di jalan.
- Robotika dalam melakukan navigasi dengan bantuan *computer vision*.
- Industri manufaktur dalam mendeteksi cacat produk secara cepat.
- Pengawasan cerdas (smart surveillance) melalui pengenalan wajah atau aktivitas manusia.
- Night Vision untuk mendeteksi objek pada kondisi pencahayaan rendah.

Convolutional Neural Network (CNN)

CNN merupakan salah satu arsitektur jaringan saraf tiruan yang paling banyak digunakan dalam pengolahan citra digital. CNN dirancang untuk mengenali pola visual secara hierarkis, mulai dari tepi (*edge*), bentuk, hingga objek yang lebih kompleks (Hidayat & Saputra, 2019). Struktur utama CNN meliputi:

1. Convolutional Layer – melakukan ekstraksi fitur dari citra dengan filter tertentu.
2. Pooling Layer – mengurangi dimensi citra tanpa menghilangkan informasi penting.
3. Fully Connected Layer – menghubungkan hasil ekstraksi fitur dengan kelas keluaran.
4. Activation Function – memberikan kemampuan non-linear, seperti ReLU dan Softmax.

CNN terbukti efektif dalam klasifikasi gambar, pengenalan objek, dan deteksi citra real-time.

TensorFlow dan Keras

TensorFlow adalah *library open-source* yang banyak digunakan untuk mengembangkan model *machine learning* dan *deep learning*. Sementara itu, Keras merupakan API tingkat tinggi yang berjalan di atas TensorFlow, sehingga memudahkan proses pembuatan model jaringan saraf tiruan dengan sintaks yang

lebih sederhana (Pratama, 2021). Kombinasi keduanya sangat mendukung praktikum maupun penelitian karena memiliki fleksibilitas tinggi, dukungan GPU, serta banyak modul siap pakai.

Computer Vision dan Real-time Detection

Computer Vision adalah bidang ilmu yang mempelajari bagaimana komputer dapat "melihat" dan memahami objek visual. Dengan integrasi CNN dan *library* seperti OpenCV, sistem dapat melakukan deteksi objek secara real-time melalui kamera (Haryanto, 2018). Teknologi ini banyak digunakan pada sistem pengawasan cerdas, robotika, hingga aplikasi *smart city*.

Night Vision dalam Deep Learning

Night Vision adalah teknik yang digunakan untuk meningkatkan visibilitas objek pada kondisi minim cahaya. Beberapa metode yang umum digunakan adalah konversi ke citra grayscale, penerapan *color mapping*, hingga pemanfaatan algoritma *deep learning* untuk peningkatan kualitas citra (Setiawan, 2020). Dalam praktikum ini, Night Vision diimplementasikan dengan pendekatan sederhana menggunakan OpenCV, yaitu mengubah citra ke grayscale kemudian menerapkan *color map*, sehingga sistem tetap dapat mendeteksi objek meskipun cahaya terbatas.

4. Analisis dan Diskusi

1. Analisis Hasil Klasifikasi CNN

Implementasi Convolutional Neural Network (CNN) pada sistem klasifikasi citra berhasil menunjukkan performa yang cukup baik. Model mampu mengenali objek pada citra yang ditangkap kamera sebagai kelas “*sea*” atau laut. Hal ini sesuai dengan karakteristik visual yang ditampilkan, yaitu adanya pola permukaan air, gradasi warna biru, serta tekstur gelombang yang menjadi ciri khas dari kelas laut.

Output yang ditampilkan melalui jendela *Frame* mengindikasikan bahwa arsitektur CNN yang digunakan telah mampu mengekstraksi fitur-fitur visual utama yang

relevan, kemudian memetakannya ke label kelas yang benar. Fakta ini membuktikan bahwa model tidak hanya “menghafal” dataset, melainkan mampu melakukan generalisasi terhadap data baru secara *real-time*.

Meskipun nilai akurasi numerik tidak ditampilkan dalam hasil praktikum, pengenalan citra laut secara tepat dapat dijadikan indikator bahwa model bekerja sesuai dengan tujuan awal. Namun demikian, untuk memastikan konsistensi kinerja dan menghindari bias pada satu kelas, perlu dilakukan uji coba lebih lanjut pada kelas lain dalam dataset, seperti *forest*, *mountain*, *street*, *buildings*, maupun *glacier*. Selain itu, variasi kondisi pencahayaan, jarak kamera, serta gangguan visual (misalnya bayangan atau objek lain di sekitar) juga penting untuk menguji robustnes model.

2. Analisis Mode Night Vision

Pada jendela *Night Vision*, citra laut diubah terlebih dahulu menjadi skala keabuan (*grayscale*), kemudian diberi pemetaan warna semu (*false color*) menggunakan metode COLORMAP_JET dari OpenCV. Transformasi ini menghasilkan citra dengan gradasi warna merah–kuning–biru yang kontras, sehingga perbedaan intensitas cahaya antara area gelap dan terang menjadi lebih jelas terlihat.

Secara visual, metode ini terbukti meningkatkan keterbacaan fitur pada citra, terutama pola gelombang dan permukaan air laut, dibandingkan dengan citra aslinya. Meskipun warna yang ditampilkan bukanlah warna nyata dari objek, informasi visual yang dihasilkan tetap membantu sistem maupun pengguna dalam mengenali pola yang sebelumnya sulit diamati karena keterbatasan pencahayaan. Dengan demikian, mode *Night Vision* dapat dianggap sebagai solusi awal yang cukup efektif untuk memperbaiki visibilitas pada kondisi pencahayaan rendah.

Namun, analisis lebih lanjut menunjukkan bahwa metode ini masih memiliki keterbatasan. Jika kondisi cahaya benar-benar gelap (misalnya malam hari tanpa cahaya buatan sama sekali), teknik ini tidak mampu menghasilkan detail yang signifikan karena sumber data visual yang sangat minim. Oleh karena itu, penerapan *Night Vision* dengan *color mapping* sebaiknya dikombinasikan dengan

metode lain, misalnya penggunaan sensor inframerah atau algoritma *deep learning* khusus untuk *low-light image enhancement*, agar hasilnya lebih natural dan dapat digunakan secara praktis.

3. Diskusi Keterkaitan Teori dan Implementasi

Secara keseluruhan, hasil yang diperoleh mendukung teori yang telah dibahas dalam landasan. CNN terbukti mampu mengekstraksi fitur hierarkis dari citra, mulai dari pola sederhana seperti tepi (*edges*) hingga pola kompleks seperti tekstur air atau bentuk gelombang. Hal ini sesuai dengan konsep dasar CNN yang memanfaatkan *convolutional layer* untuk mengenali fitur lokal dan *fully connected layer* untuk klasifikasi akhir.

Selain itu, penggunaan OpenCV sebagai media integrasi real-time terbukti efektif. OpenCV tidak hanya berfungsi sebagai media akuisisi citra dari kamera, tetapi juga mendukung proses visualisasi hasil klasifikasi serta pengolahan tambahan seperti mode *Night Vision*. Hal ini memperlihatkan fleksibilitas OpenCV dalam menggabungkan teori CNN dengan aplikasi nyata.

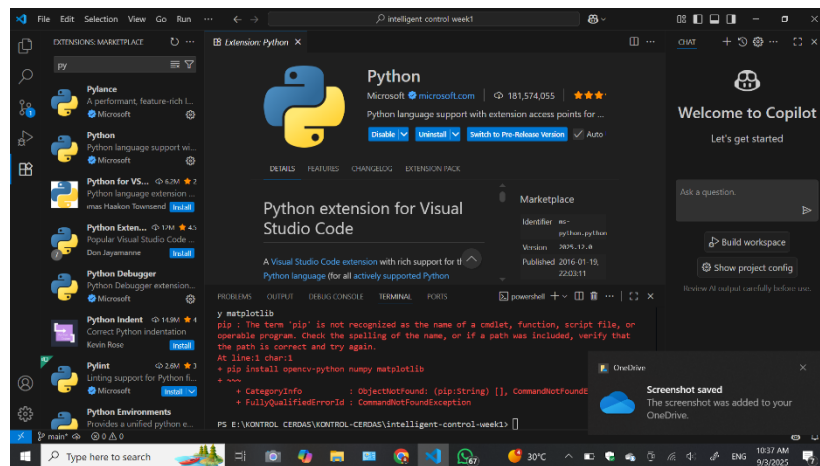
Di sisi lain, diskusi juga menyoroti beberapa catatan penting:

1. Kinerja Model CNN Model perlu diuji lebih lanjut pada berbagai kelas dan kondisi untuk mengetahui sejauh mana kemampuannya melakukan generalisasi. Hal ini penting agar sistem tidak hanya bekerja baik pada kelas “sea”, tetapi juga konsisten pada kelas lain yang memiliki fitur visual berbeda.
2. Performa Real-Time Implementasi real-time pada perangkat tanpa GPU biasanya menimbulkan penurunan *frame per second* (FPS). Kondisi ini dapat berpengaruh pada aplikasi nyata, misalnya dalam sistem navigasi atau monitoring. Oleh karena itu, optimasi model melalui teknik *model compression*, *quantization*, atau penggunaan arsitektur ringan seperti *MobileNet* bisa menjadi solusi.
3. Pengembangan Night Vision Metode sederhana dengan *color mapping* sudah memberikan hasil yang baik, namun masih jauh dari optimal jika

digunakan pada kondisi pencahayaan ekstrem. Penerapan metode lanjutan berbasis *deep learning* seperti *Low-Light Image Enhancement (LLIE)* atau penggunaan *Generative Adversarial Networks (GAN)* dapat meningkatkan kualitas citra dengan mempertahankan detail asli.

Dengan demikian, implementasi yang dilakukan tidak hanya menunjukkan bahwa teori CNN dan pengolahan citra dapat direalisasikan, tetapi juga membuka peluang pengembangan sistem lebih lanjut, baik dalam aspek akurasi klasifikasi, efisiensi komputasi, maupun peningkatan kualitas visual pada kondisi lingkungan nyata.

Kendala dan Solusi



Kendala yang muncul adalah saat instalasi Python versi 3.11.6, di mana opsi “Add Python to PATH” tidak dicentang sehingga perintah pip tidak dikenali di terminal. Solusinya adalah melakukan instal ulang Python dan memastikan semua opsi, termasuk “Add Python to PATH”, dicentang sehingga perintah pip dapat berjalan dengan baik dan instalasi library berhasil dilakukan.

5. Assignment

Pada praktikum ini kami membuat sebuah sistem pendeteksi gambar berbasis Convolutional Neural Network (CNN) yang dapat mengenali objek-objek pemandangan seperti laut (*sea*), hutan (*forest*), jalan (*street*), bangunan (*buildings*), gunung (*mountain*), dan lain-lain. Program ini dilatih menggunakan dataset Intel Image Classification, kemudian diintegrasikan dengan OpenCV untuk melakukan prediksi secara real-time melalui kamera. Selain itu, ditambahkan pula mode Night Vision untuk meningkatkan visibilitas citra dalam kondisi pencahayaan rendah.

Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini:

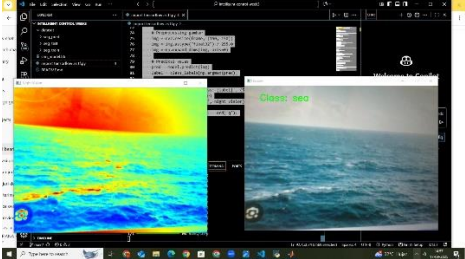
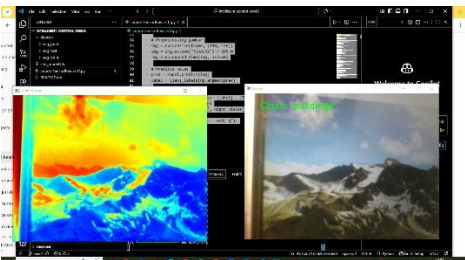
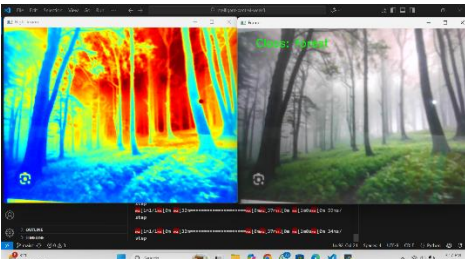
Bagian Kode	Deskripsi Fungsi	Analisis / Kegunaan
<code>import tensorflow as tf ... import cv2, numpy as np</code>	Import library utama	TensorFlow & Keras untuk CNN, OpenCV untuk akuisisi citra real-time, NumPy untuk manipulasi array.
Bagian 1: Preprocessing Data		
<code>train_dir & val_dir</code>	Menentukan direktori dataset training dan validasi	Struktur folder dataset harus sesuai format ImageDataGenerator (tiap folder = nama kelas).
<code>ImageDataGenerator(...)</code>	Membuat generator data dengan augmentasi	<code>train_datagen</code> : augmentasi (rotasi, zoom, flip). <code>val_datagen</code> : hanya normalisasi. Membantu mengurangi <i>overfitting</i> .
<code>flow_from_directory(...)</code>	Membaca gambar dari folder dataset	Mengubah dataset menjadi batch untuk training CNN. Ukuran gambar diubah ke (150, 150).
Bagian 2: Training Model CNN		
<code>Sequential([...])</code>	Definisi arsitektur CNN	CNN terdiri dari Conv2D + MaxPooling (ekstraksi fitur), Flatten + Dense (klasifikasi), dan Dropout (mencegah <i>overfitting</i>).
<code>Conv2D(32, (3, 3)) → MaxPooling2D</code>	Layer konvolusi dan pooling pertama	Mengekstraksi fitur dasar (tepi, pola sederhana). Pooling mengecilkan dimensi.
<code>Conv2D(64, (3, 3)) → MaxPooling2D</code>	Layer konvolusi dan pooling kedua	Mengekstraksi fitur lebih kompleks (tekstur, bentuk).
<code>Flatten()</code>	Mengubah hasil ekstraksi fitur jadi 1D	Dibutuhkan agar bisa diproses Dense layer.
<code>Dense(128, activation='relu')</code>	Fully connected layer	Menggabungkan fitur untuk klasifikasi.

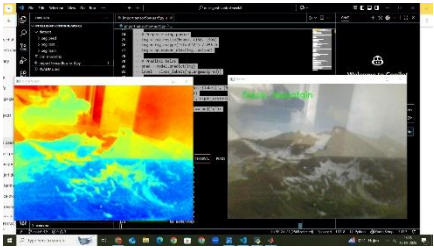
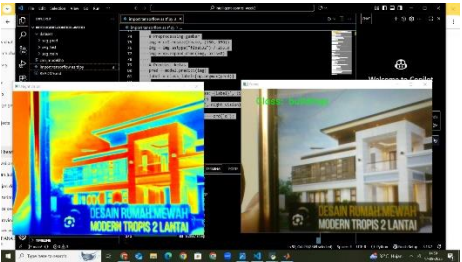
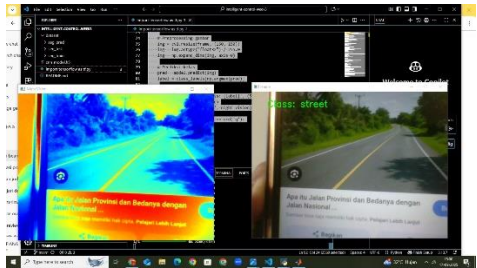
<code>Dropout(0.5)</code>	Menonaktifkan 50% neuron saat training	Mencegah model terlalu menghafal data (<i>overfitting</i>).
<code>Dense(6, activation='softmax')</code>	Output layer	Klasifikasi ke 6 kelas (karena dataset Intel Image Classification punya 6 kategori).
<code>model.compile(...)</code>	Menentukan optimizer, loss, dan metric	Optimizer: Adam, Loss: categorical crossentropy, Metric: akurasi.
<code>model.fit(...)</code>	Melatih model CNN	Melakukan training dengan data augmentasi (train) dan validasi (val).
<code>model.save('cnn_model.h5')</code>	Menyimpan model terlatih	Agar bisa digunakan kembali tanpa training ulang.
Bagian 3: Integrasi dengan OpenCV		
<code>load_model('cnn_model.h5')</code>	Memanggil model CNN yang sudah dilatih	Digunakan untuk prediksi real-time.
<code>class_labels = list(train_generator.class_indices.keys())</code>	Mendapatkan label kelas	Agar prediksi angka (0–5) bisa dikonversi ke nama kelas.
<code>cv2.VideoCapture(0)</code>	Mengaktifkan kamera laptop/webcam	Untuk mengambil input citra secara langsung.
<code>night_vision = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)</code>	Mengubah citra ke grayscale	Tahap awal mode <i>Night Vision</i> .
<code>cv2.applyColorMap(..., cv2.COLORMAP_JET)</code>	Memberi warna semu (false color)	Meningkatkan visibilitas area gelap.
<code>cv2.resize(frame, (150, 150))</code>	Menyesuaikan ukuran citra input	Harus sama dengan ukuran input model CNN.
<code>img.astype("float32")/255.0</code>	Normalisasi citra	Menyelaraskan dengan preprocessing dataset.
<code>np.expand_dims(img, axis=0)</code>	Menambahkan dimensi batch	Bentuk input jadi (1, 150, 150, 3) agar cocok dengan model.
<code>model.predict(img)</code>	Prediksi kelas citra dari kamera	Hasil berupa probabilitas tiap kelas.

<code>np.argmax(pred)</code>	Mengambil kelas dengan probabilitas tertinggi	Output = kelas dengan skor prediksi paling besar.
<code>cv2.putText(frame, ...)</code>	Menampilkan hasil klasifikasi pada jendela	Label hasil klasifikasi ditulis di atas video.
<code>cv2.imshow('Frame', frame)</code>	Menampilkan citra asli dengan label	Jendela utama prediksi CNN.
<code>cv2.imshow('Night Vision', night_vision)</code>	Menampilkan citra mode <i>Night Vision</i>	Jendela tambahan untuk visibilitas rendah.
<code>cv2.waitKey(1)</code>	Kontrol untuk keluar dengan tombol q	Jika ditekan q , loop berhenti.
<code>cap.release(), cv2.destroyAllWindows()</code>	Menutup kamera dan jendela OpenCV	Membersihkan resource setelah selesai.

6. Data dan Output Hasil Pengamatan

Setelah dilakukan pengujian terhadap program yang telah dimodifikasi, pengujian deteksi warna menggunakan metode Support Vector Machine (SVM), sistem berhasil mengidentifikasi warna pada objek yang ditangkap kamera secara real-time. Setiap objek diberi bounding box dengan label warna sesuai hasil prediksi, serta nilai akurasi deteksi ditampilkan pada layar. Dari hasil tersebut terlihat bahwa sistem mampu mengenali warna dengan cukup baik, meskipun tingkat akurasi masih dipengaruhi oleh faktor pencahayaan dan kemiripan warna dengan latar belakang. seperti yang ditunjukkan pada tabel hasil pengamatan berikut.

No	Input Citra (Kamera)	Output Night Vision	Hasil Prediksi CNN	Analisis	Gambar
1	Gambar laut (permukaan air & gelombang)	Citra berwarna semu dengan colormap JET (biru–kuning–merah) yang menonjolkan perbedaan intensitas pada permukaan laut	Class: sea	Model berhasil mengenali laut dengan baik. Fitur gelombang dan pola tekstur air diekstraksi dengan tepat. Night Vision membantu menonjolkan kontras meski warna asli hilang.	
2	Gambar pegunungan bersalju (dengan langit di atasnya)	Citra colormap JET menampilkan gradasi biru–kuning–merah pada area gunung dan langit	Class: buildings	Model mengalami <i>misclassification</i> , citra pegunungan dikenali sebagai <i>buildings</i> . Hal ini kemungkinan karena tekstur dan garis horizon menyerupai pola bangunan pada dataset. Perlu peningkatan akurasi dengan menambah data pelatihan khusus.	
3	Gambar jalan dengan pepohonan di sekitarnya	Citra colormap JET menonjolkan bentuk jalan dan vegetasi sekitar dengan warna semu	Class: forest	Model mengenali objek sebagai <i>forest</i> . Hasil ini cukup relevan karena terdapat vegetasi dominan di sekitar jalan, walaupun seharusnya bisa juga diklasifikasikan sebagai <i>street</i> . Menunjukkan bahwa model	

				masih kesulitan membedakan kelas dengan kemiripan fitur.	
	Gambar pegunungan (puncak dengan salju/awan)	Citra berwarna semu dengan colormap JET (merah–kuning–biru) yang mempertegas perbedaan kontur permukaan gunung	Class: mountain	Model berhasil mengenali gunung dengan tepat. Ciri utama seperti bentuk puncak dan tekstur alami berhasil diekstraksi. Night Vision membantu menampilkan perbedaan elevasi meskipun kondisi cahaya kurang jelas.	
	Gambar rumah modern bertingkat (bangunan)	Citra berwarna semu dengan colormap JET yang menyoroti garis tepi bangunan dan struktur arsitektur	Class: buildings	Prediksi sesuai dengan objek sebenarnya. CNN mendeteksi fitur bangunan dari pola garis lurus, sudut, dan struktur simetris. Night Vision memperjelas kontras antara dinding dan jendela.	
	Gambar jalan raya lurus dengan marka jalan kuning	Citra berwarna semu dengan colormap JET yang memperlihatkan kontras jelas antara permukaan jalan dan pepohonan sekitar	Class: street	Model mengenali jalan dengan benar. Pola marka jalan lurus dan perspektif jalan menjadi fitur dominan. Night Vision membantu menyoroti bentuk jalur meskipun warna alami tidak terlihat.	

7. Kesimpulan

Berdasarkan hasil implementasi, dapat disimpulkan bahwa model Convolutional Neural Network (CNN) yang digunakan mampu mengenali objek citra secara real-time dengan cukup baik. Hal ini terlihat dari keberhasilan klasifikasi pada kelas tertentu seperti *sea* dan *forest*. Namun, masih terdapat beberapa kesalahan prediksi, misalnya citra gunung yang dikenali sebagai *buildings*, yang menunjukkan bahwa model masih mengalami kesulitan dalam membedakan kelas dengan kemiripan fitur visual. Penerapan mode Night Vision menggunakan colormap JET juga terbukti membantu meningkatkan visibilitas citra pada kondisi pencahayaan rendah, sehingga detail objek lebih mudah diamati meskipun tidak berpengaruh langsung pada hasil klasifikasi CNN. Secara keseluruhan, sistem ini menunjukkan potensi untuk diaplikasikan pada pendeteksian objek lingkungan seperti laut, hutan, jalan, maupun bangunan, meskipun akurasi masih perlu ditingkatkan.

8. Saran

Adapun saran yang dapat diberikan adalah perlunya perluasan dataset dengan variasi citra yang lebih beragam agar model dapat lebih baik dalam membedakan objek yang memiliki kemiripan fitur. Selain itu, arsitektur CNN dapat ditingkatkan atau menggunakan metode *transfer learning* agar hasil klasifikasi lebih akurat. Dari sisi performa, penggunaan GPU atau model ringan seperti TensorFlow Lite dapat membantu mempercepat prediksi pada perangkat dengan spesifikasi terbatas. Pengembangan mode Night Vision juga dapat ditingkatkan dengan metode *low-light enhancement* berbasis deep learning sehingga citra lebih natural dan tetap jelas meskipun dalam kondisi gelap total. Terakhir, pengujian lebih lanjut dengan berbagai kondisi pencahayaan, sudut pengambilan gambar, dan jenis objek yang berbeda perlu dilakukan untuk mengetahui sejauh mana kemampuan sistem dalam penerapan nyata.

9. Daftar Pustaka

D. Pramadihanto, M. I. Fahmi, dan A. W. Setiawan, "Pengenalannya Objek dan Warna untuk Robot Pemilah Barang Berbasis Computer Vision," *Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 4, no. 2, pp. 87-95, Agu. 2018, doi: 10.14710/jitsi.4.2.2018.87-95